

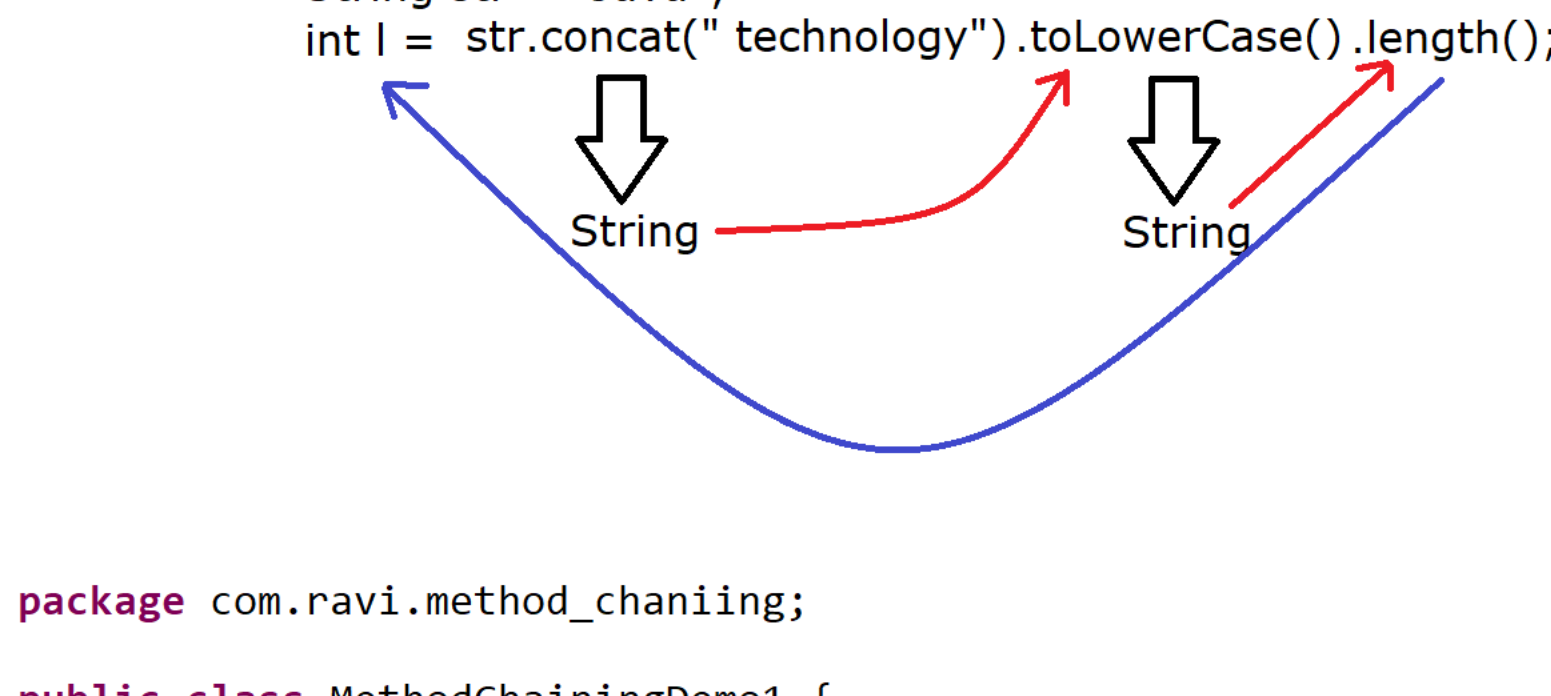
## What is Method Chaining in Java ?

It is a technique through which we can **multiple methods** in a single statement OR in a single line.

When we want to call another method upon existing method then we need to depend upon **existing method return type that means based on the method return type we can call next method.**

In method chaining the last method call will decide the final return type of the method.

Example :



```
package com.ravi.method_chaining;

public class MethodChainingDemo1 {

    public static void main(String[] args)
    {
        String s1 = "java";
        int l = str.concat(" technology").toLowerCase().length();
        IO.println("length is :"+length);

        IO.println(".....");

        String s2 = "India";
        boolean x = s2.toLowerCase().concat(" is a great country").startsWith("i");
        IO.println(x);

        IO.println(".....");

        char gender = IO.readLine("Enter your Gender [M/F] :").charAt(0);
        IO.println("Your Gender is :"+gender);
    }
}
```

## Object class and its methods :

\* java.lang.Object is the super class of all the classes (predefined + user-defined) in java.

\* It contains 11 non static methods.

\* Since Object is the super class so, we can override all the non final methods of Object class to any sub class in java.

Working with Object class methods :

public native final **java.lang.Class** getClass() :

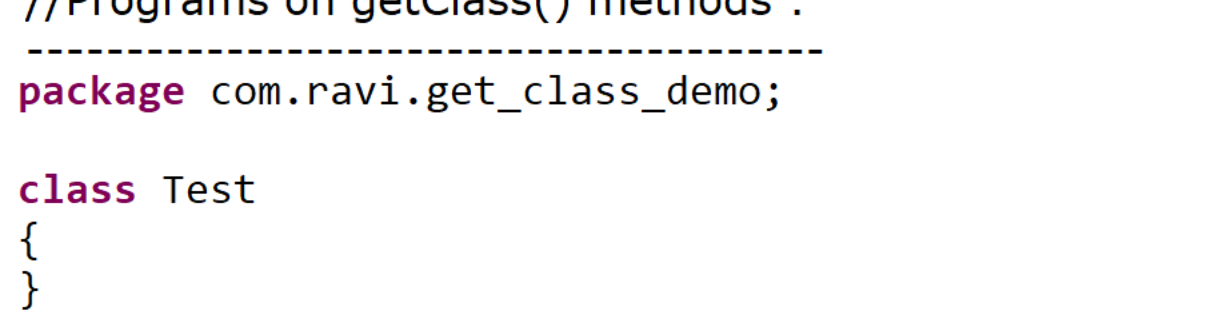
\* Predefined non static method of Object class.

\* This method is useful to **get the runtime object information of a class.**

\* This method return type is java.lang.Class

\* Actually this method will return -> **class keyword + Fully Qualified Name** (Package Name + class name)

\* The return type of this method is, **java.lang.Class so, according to method chaining concept further we can call any method from java.lang.Class**



\* getName() is a predefined non static method of java.lang.Class which will provide only FQN (Fully Qualified name) and return type of this method is String.

public String getName()

//Programs on getClass() methods :

```
package com.ravi.get_class_demo;

class Test
{
}

public class GetClassDemo1 {

    public static void main(String[] args)
    {
        Test t1 = new Test();
        IO.println(t1.getClass());

        String str = "Java";
        IO.println(str.getClass());
    }
}

package com.ravi.get_class_demo;

class Animal
{
}

class Dog extends Animal{}

public class GetClassDemo2 {

    public static void main(String[] args)
    {
        Animal a = new Dog();
        IO.println(a.getClass());
    }
}

package com.ravi.get_class_demo;

class Alpha
{
}

public class GetClassDemo3 {

    public static void main(String[] args)
    {
        Alpha a = new Alpha();
        String fqcn = a.getClass().getName();
        IO.println(fqcn);
    }
}

package com.ravi.get_class_demo;

class Beta
{
}

public class GetClassDemo4 {

    public static void main(String[] args)
    {
        Beta b = new Beta();
        IO.println(b.getClass()); //class + FQN
        IO.println(b.getClass().getName()); //FQN
    }
}

}
```

## public native int hashCode() :

\* Predefined non static method of Object class.

\* It is used to find out the **hashcode value of an object**, this hashcode value is **very useful** to find the bucket location in the **hashtable data structure** in the HashMap class of Collections Framework.

\* It is not **meant for Object comparison**, **It is used to provide hashcode value of a particular object for the benefit of Hashtable Data structure.**

//Program

```
package com.ravi.hash_code;

public class HashCodeDemo1
{
    public static void main(String[] args)
    {
        Integer i = 65;
        String s = "A";

        IO.println(i.hashCode());
        IO.println(s.hashCode());
    }
}

package com.ravi.hash_code;

class Student
{
}

public class HashCodeDemo2 {

    public static void main(String[] args)
    {
        Student s1 = new Student();
        int hash = s1.hashCode();
        IO.println(hash);
    }
}

3) public String toString()

* Predefined non static method of Object class.
* It is used to represent object in a string format (String representation of the object)
* It is used to represent all our non static fields in the String format so, the information is concise hence anyone can easily read the object data in String format.
The internal logic of toString() method written by Java :

public String toString()
{
    return getClass().getName()+"@"+ Integer.toHexString(hashCode());
}

Note :
1) toString() method of Object class internally uses getClass() & hashCode() method of Object class.
2) toHexString() is a predefined static method of Integer class which will take int as a parameter and return the hexadecimal value of the given integer in the String format.

    public static String toHexString(int hashCode)

package com.ravi.to_string;

class Employee
{
}

public class ToStringDemo1
{
    public static void main(String[] args) {

        Employee e1 = new Employee();
        IO.println("Hash code value ");
        int hash = e1.hashCode();
        IO.println(hash);
        IO.println(Integer.toHexString(hash)); //2a84aee7

        IO.println("To String method value");

        IO.println(e1); //com.ravi.to_string.Employee@2a84aee7
        //com.ravi.to_string.Employee@2a84aee7
    }
}

package com.ravi.to_string;

class Employee
{
    public String toString()
    {
        return "Employee object representation in String format";
    }
}

public class ToStringDemo2
{
    public static void main(String[] args) {

        Employee e1 = new Employee();
        IO.println(e1);
    }
}

4) public boolean equals(Object obj)

* Predefined non static method of Object class.
```